

SIGN TRACK



INDRAPRASTHA INSTITUTE *of*
INFORMATION TECHNOLOGY
DELHI

Group Members:

VEER AMOL SINGH
ANANY SINGH DIVY
ANIRUDH SRINIVAS



Problem Statement and Scope



Problem Statement :

Develop a real-time Traffic Sign Detection system using YOLOv8 to enhance road safety and support autonomous navigation. The system aims to accurately detect and recognize traffic signs in images and videos using deep learning.

Scope:

Input: Traffic sign images & video streams from dashcams, surveillance cameras, or vehicle sensors.

Dataset: 4,969 samples, split into Train, Validation, and Test sets.

Output: Annotated Images/video frames with bounding boxes, labels, and confidence scores.

Target Users:

Autonomous Vehicles & ADAS Systems: For real-time sign recognition, ensuring compliance with traffic rules.

Traffic Monitoring Authorities: To track violations using CCTV & IoT-based monitoring.

Urban Planners & Smart City Developers: To analyze traffic compliance and improve infrastructure planning.

Related Work



1. **SVM with Histogram of Oriented Gradients (HOG):** Uses HOG to extract shape & edge-based features. Classified using Support Vector Machine (SVM) for robust detection.
Reference: Nabil Ahmed et al. - High-precision SVM-HOG model for Bangladeshi traffic signs.

1. **Convolutional Neural Networks (CNNs) :** Learns hierarchical features directly from images. Effective for real-time traffic sign recognition.
Reference: Deepali Patil et al. - CNN-based system that captures and processes traffic signs in video

1. **Hybrid GIS & Machine Learning Approaches:** Combines geo-tagged images with ML for sign detection & positioning.
Reference: Zihao Wu - Uses Google Street View & GoPro for mapping traffic signs.

Baseline Methods and comparison



Baseline Models for comparison

To evaluate YOLO v8, we compare against:

- **SVM + HOG:** Traditional machine learning benchmark for traffic sign detection.
- **CNN-based detection:** Standard deep learning approach widely used in image recognition.
- **Hybrid GIS Models:** Combine geographic information systems with computer vision to incorporate spatial context.

Identified Gaps in Existing Approaches:

- **SVM + HOG:** Limited generalization under varying lighting conditions and occlusions.
- **CNN-based Models:** High accuracy but computationally expensive, making them less suitable for real-time applications.
- **Hybrid GIS Models:** Require extensive geo-tagged data, which limits scalability and adaptability to new environments

Datasets and Evaluation Metrics



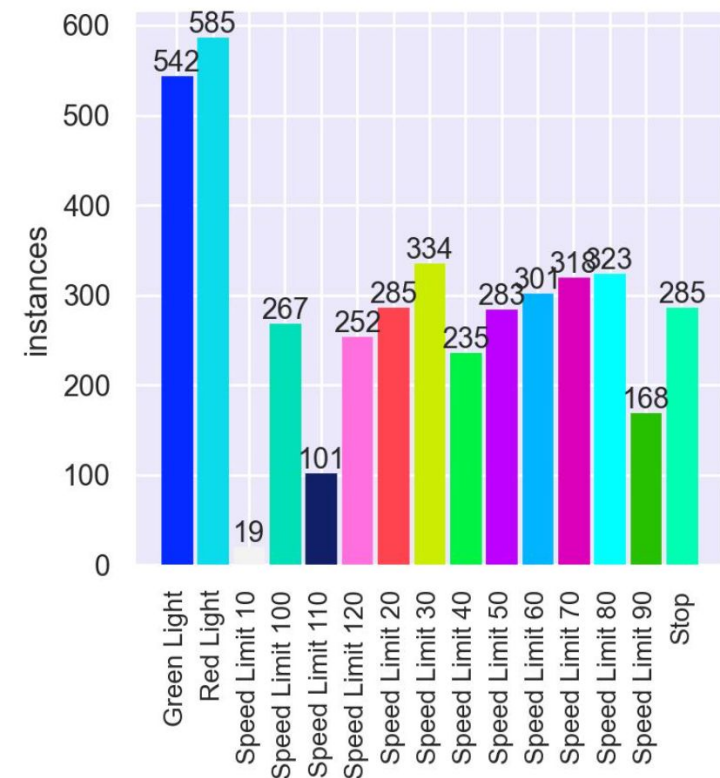
Traffic Sign Detection Dataset:

Contains images of traffic signs and surrounding environments. Used for initial model training and validation of YOLOv8. Helps assess preliminary detection performance.

Evaluation Metrics:

We ran two models on the same dataset:

Metrics	YOLOv8	YOLO11
Precision	95.50%	93.20%
Recall	83.84%	85.15%
mAP@0.5	92.58%	93.28%
mAP@0.5-0.95	79.01%	79.14%



Web Application Architecture



A client-server architecture was adopted for the traffic sign video application:

- **Backend:** A RESTful API was developed using the **Node.js** runtime and the **Express** framework. Its primary responsibilities are handling incoming video upload requests, spawning a Python child process to execute the YOLO model inference, and streaming the processed output video.
- **Frontend:** A dynamic and interactive user interface was built using the **React** JavaScript library (via Vite). It allows users to interact with the system by selecting expected label classes, uploading video files, and viewing the granular timeline and annotated results.
- **Communication:** The frontend communicates with the backend via HTTP requests. Video data is sent to the backend API as FormData using multipart/form-data. The backend streams back the output .mp4 and returns the processed detection details over a structured JSON format.

Backend API Implementation



The Node.js backend serves as the core logic hub, primarily managing I/O and orchestration:

- **Video Uploading & Storage:** The API endpoint (POST /api/videos/process) utilizes Multer to automatically validate, accept, and securely write incoming video streams (.mp4, .mkv, etc.) to local disk storage using unique identifiers.
- **Model Inference Subprocess:** Instead of executing ML code within Node.js, an inferenceService bridges the gap to Python. It dynamically spawns an independent Python subprocess (infer.py) containing the Ultralytics YOLO logic.
 1. The service passes CLI parameters for paths and thresholds.
 2. The Python subprocess initializes the fine-tuned YOLO model (best.pt) and performs frame-by-frame object detection.
 3. OpenCV draws bounding boxes and the final video is written asynchronously.
 4. Python outputs JSON statistics on stdout, which Node.js parses.
- **Video Streaming:** Instead of sending bulky base64 data, the Express server provides optimized HTTP range requests (GET /api/videos/output/:filename) to seamlessly stream the finalized, annotated MP4 back to the React client's video player.

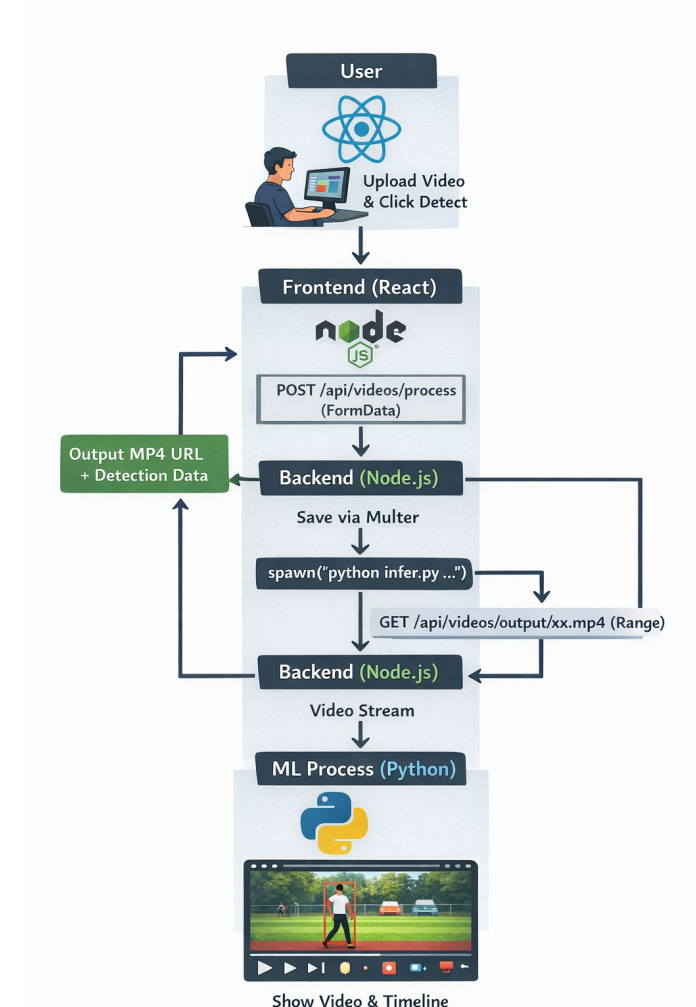
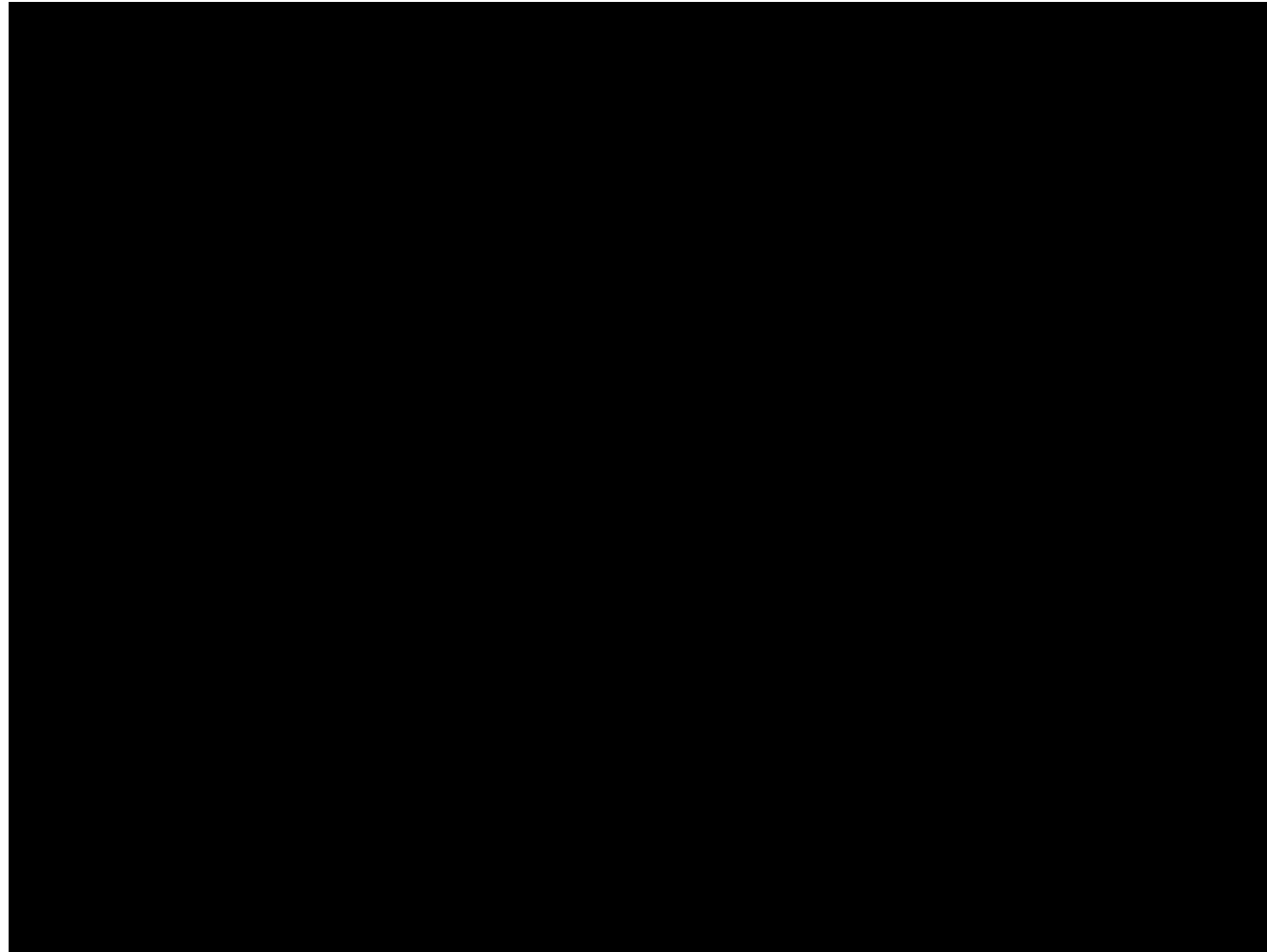
Frontend User Interface



The frontend, located in `frontend/src/`, provides the user interaction layer and orchestrates the experience:

- **User Interaction:** It presents a modern, polished interface allowing users to dynamically filter traffic sign labels (e.g., "Speed Limit 50", "Stop Sign", "Traffic Light") and provides an intuitive drag-and-drop mechanism for large video files.
- **API Communication:** Upon user action, the frontend's API client utilizes the native fetch API to post the video file. It also tracks granular uploading progress indicators for a smooth perceived user experience.
- **Result Display:** It receives a JSON payload mapping detection classes to video timestamps and dynamically updates the UI. Rather than just a static view, a `SignsTimeline` component visualizes precisely when and where a traffic sign was detected within the footage.
- **Build Process:** The React code is built into highly optimized, minified static HTML, CSS, and JS assets using Vite (`npm run build`).

Working Demo of the Project



Main slide deck ends here



Sample Dataset



Image 1



Image 2



Image 3



Image 4



Image 5



Image 6



Image 7



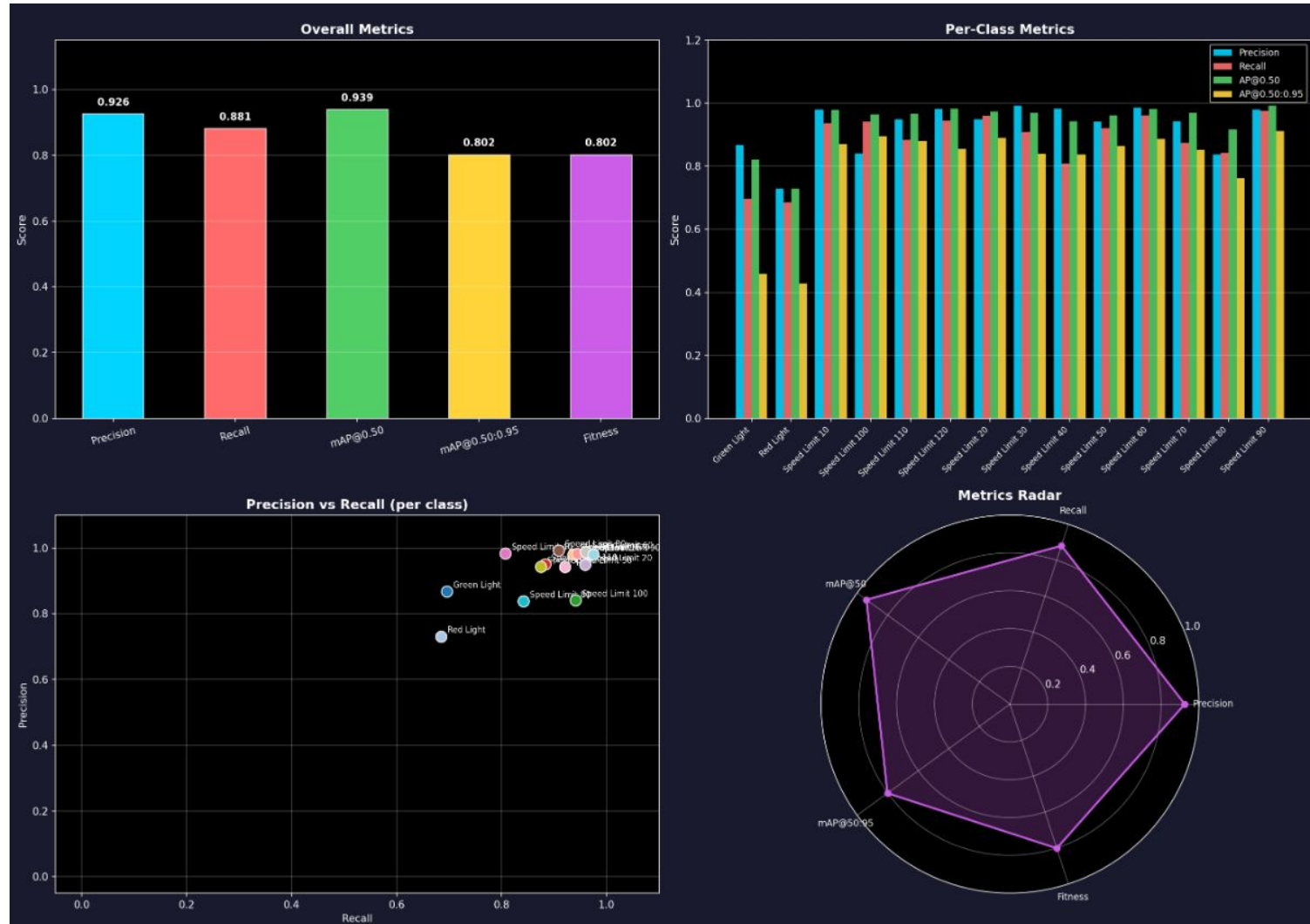
Image 8



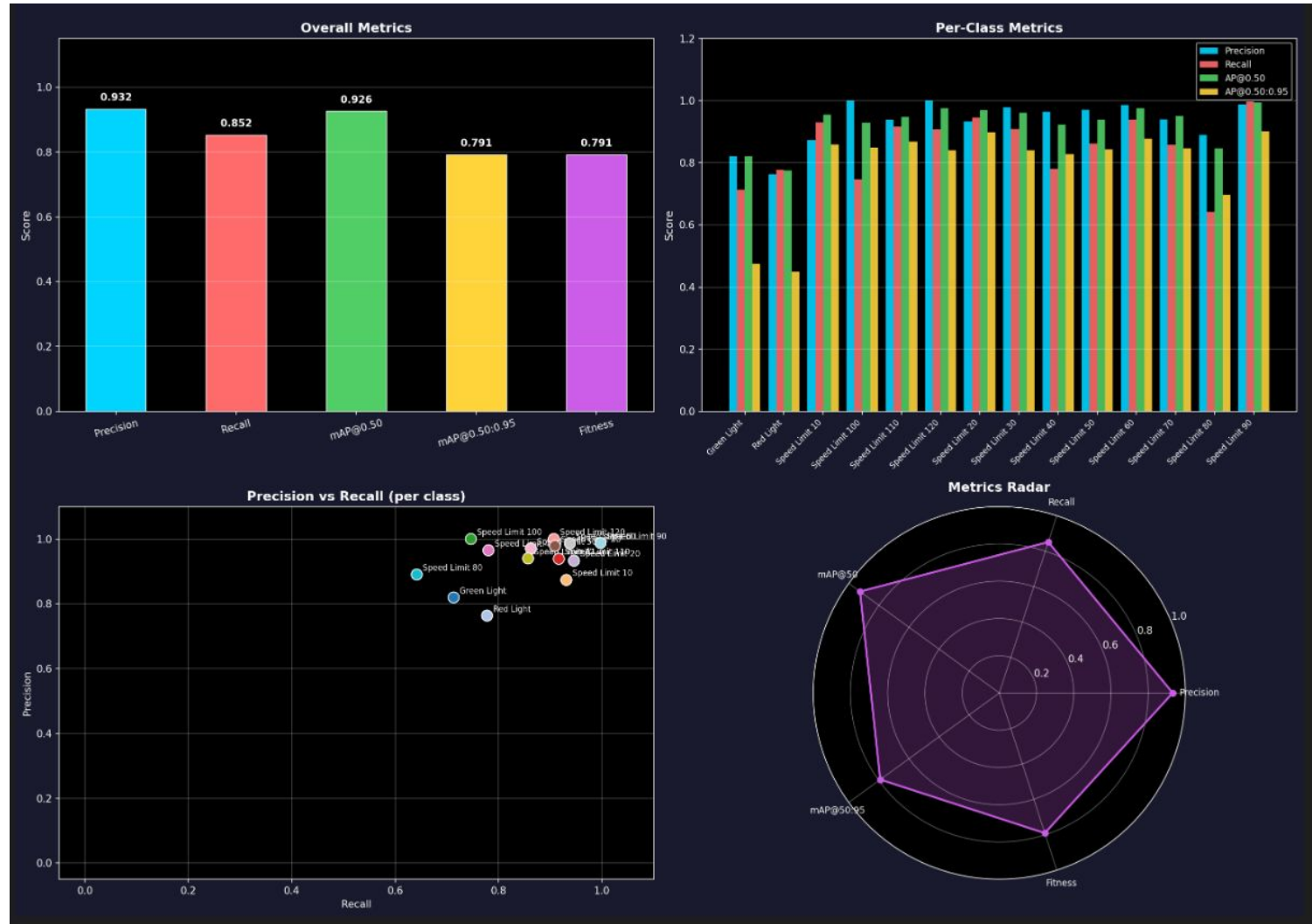
Image 9



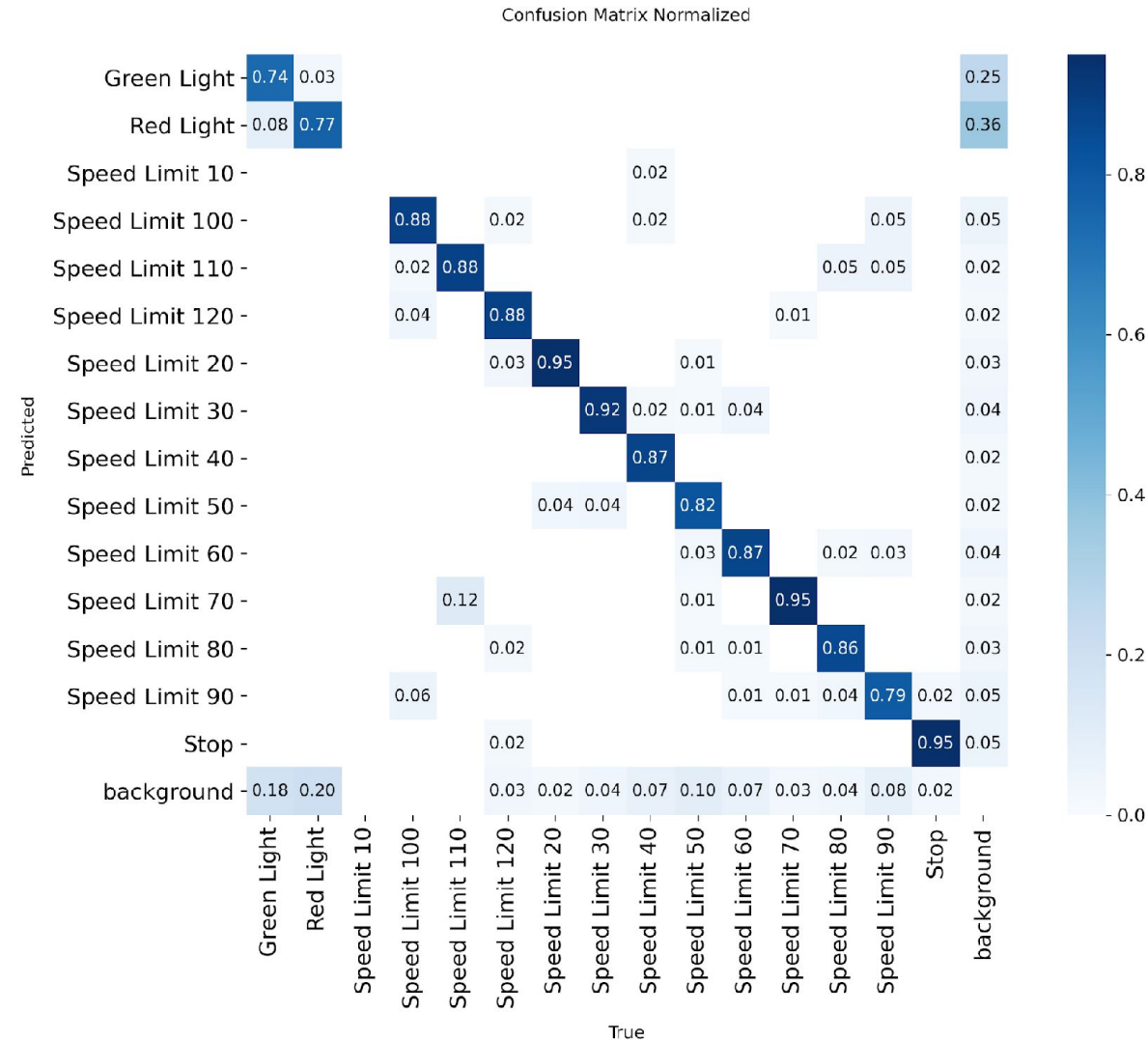
Overall metrics for YOLOv8



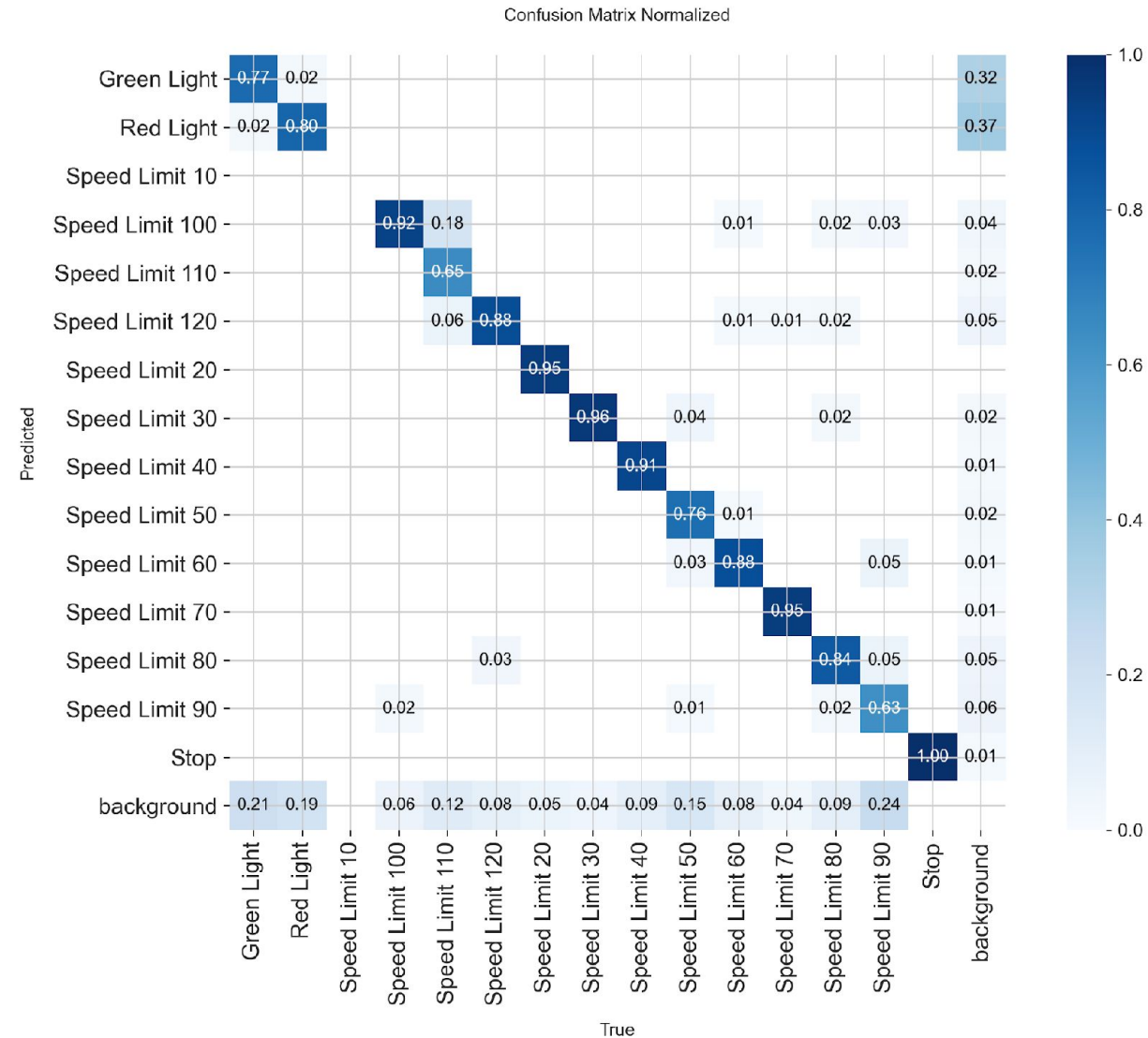
Overall metrics for YOLO11



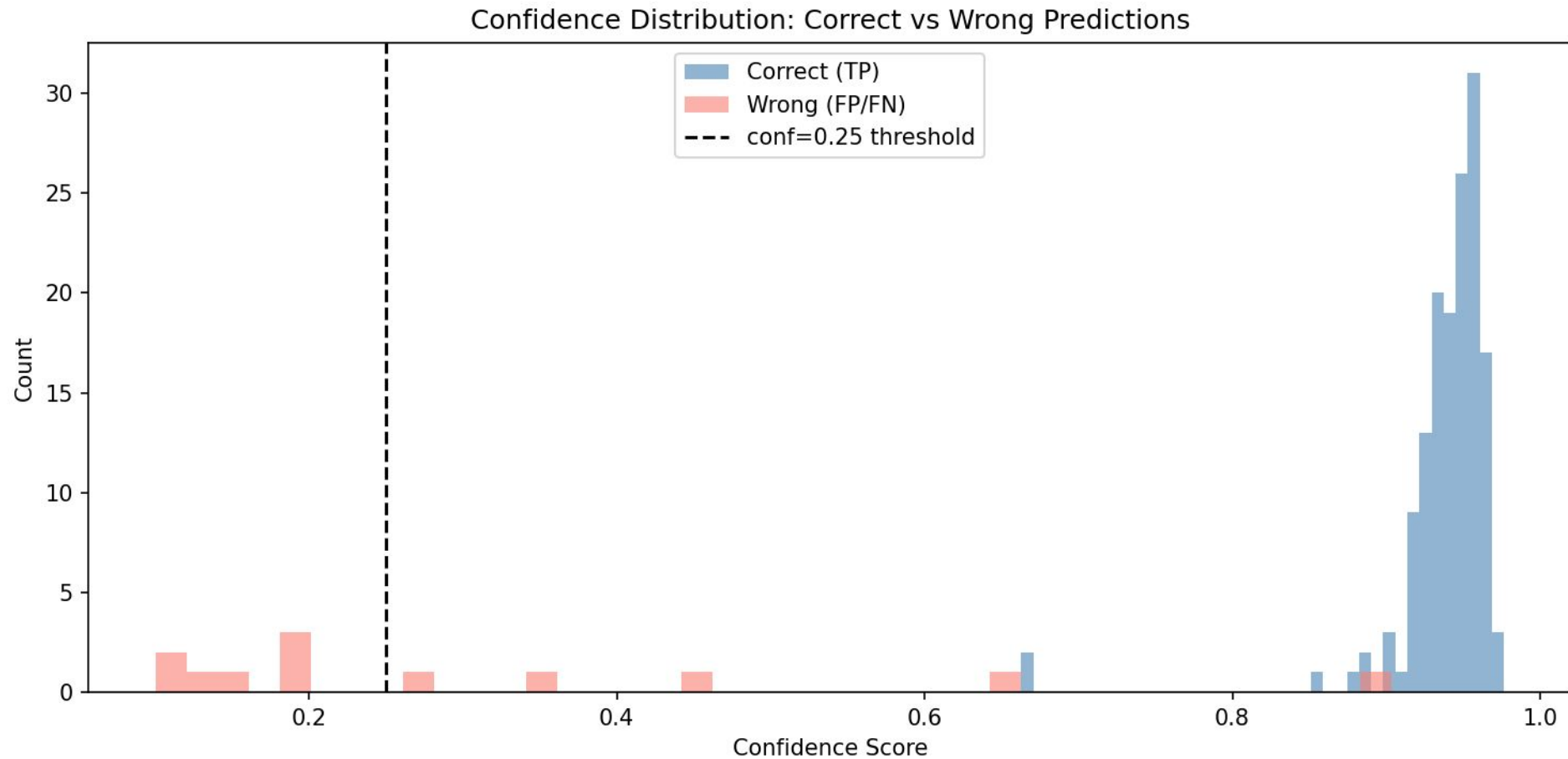
Confusion Matrix for YOLOv8



Confusion Matrix for YOLO11



Confidence Distribution for YOLOv8



Confidence Distribution for YOLO11

